

1. Ізольована популяція (варіант 2)

$dP/dt = 0,004 \cdot P \cdot (P - 180)$; $P(0) = 250$ та $P(0) = 120$; знайти $P(1)$, побудувати графіки.

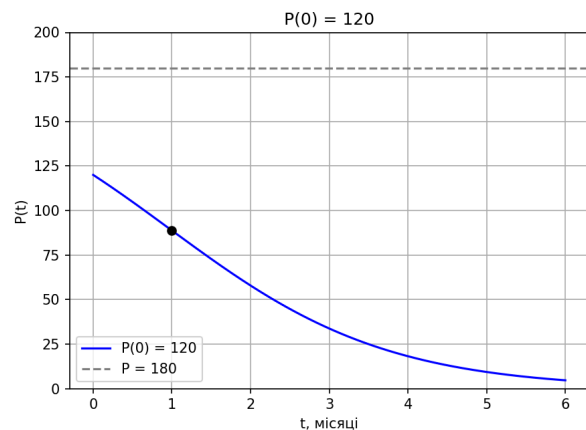
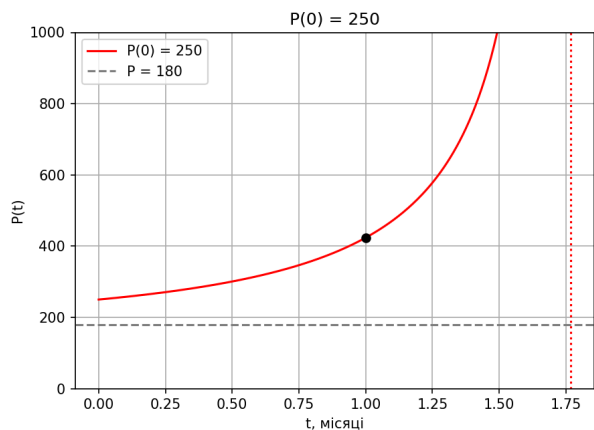
Рівняння має вигляд $dP/dt = rP(P - K)$, $r = 0,004$, $K = 180$. Розділивши змінні та використавши $P(0) = P_0$, отримуємо (формула Верхульста з $\varepsilon = -rK = -0,72$):

$$P(t) = K \cdot P_0 / (P_0 + (K - P_0) \cdot \exp(0,72t)).$$

Стационарні точки: $P = 0$ (стійка) і $P = 180$ (нестійка).

- 1) **$P_0 = 250$:** $P(t) = 45000 / (250 - 70 \cdot \exp(0,72t))$. Знаменник дорівнює 0 при $t^* = \ln(250/70)/0,72 \approx 1,77$ міс., тобто чисельність необмежено зростає за скінченний час. $P(1) \approx 424$.
- 2) **$P_0 = 120$:** $P(t) = 21600 / (120 + 60 \cdot \exp(0,72t)) \rightarrow 0$ при $t \rightarrow \infty$, популяція вимирає. $P(1) \approx 89$.

Отже, 180 – критичний поріг: якщо початкова чисельність більша, популяція зростає, якщо менша – вимирає.



Фрагмент коду і результат його запуску:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Задача 1. Ізольована популяція, варіант 2
#  $dP/dt = 0.004 * P * (P - 180)$ ,  $P(0) = 250$  та  $P(0) = 120$ 
r = 0.004
K = 180

# Стационарні точки:  $dP/dt = 0 \Rightarrow P = 0$  і  $P = 180$ 
# при  $P < 180$  похідна від'ємна (популяція спадає), при  $P > 180$  додатна (зростає)
# тобто  $P = 0$  стійка,  $P = 180$  нестійка
print("Стационарні точки: P = 0 (стійка), P = 180 (нестійка)")

# Розв'язок (відокремлення змінних, початкова умова  $P(0) = P_0$ ):
#  $P(t) = K * P_0 / (P_0 + (K - P_0) * \exp(r * K * t))$ 
def P(t, P0):
    return K * P0 / (P0 + (K - P0) * np.exp(r * K * t))

# чисельність при t = 1
for P0 in (250, 120):
    print("P0 =", P0, " P(1) =", round(P(1, P0), 2))

# при  $P_0 = 250$  знаменник перетворюється в нуль – чисельність зростає до нескінченності
t_end = np.log(250 / (250 - K)) / (r * K)
print("для P0 = 250 розв'язок іде в нескінченність при t =", round(t_end, 3))

# перевірка: розв'язуємо рівняння чисельно і порівнюємо з формулою
for P0 in (250, 120):
    sol = solve_ivp(lambda t, p: r * p * (p - K), [0, 1], [P0], rtol=1e-10)
    print("перевірка чисельно, P0 =", P0, " P(1) =", round(sol.y[0][-1], 2))
```

```
dunnaya@MacBook-Air-Dana mod1 % python3 task1.py
Стационарні точки: P = 0 (стійка), P = 180 (нестійка)
P0 = 250 P(1) = 423.77
P0 = 120 P(1) = 88.79
для P0 = 250 розв'язок іде в нескінченність при t = 1.768
перевірка чисельно, P0 = 250 P(1) = 423.77
перевірка чисельно, P0 = 120 P(1) = 88.79
```

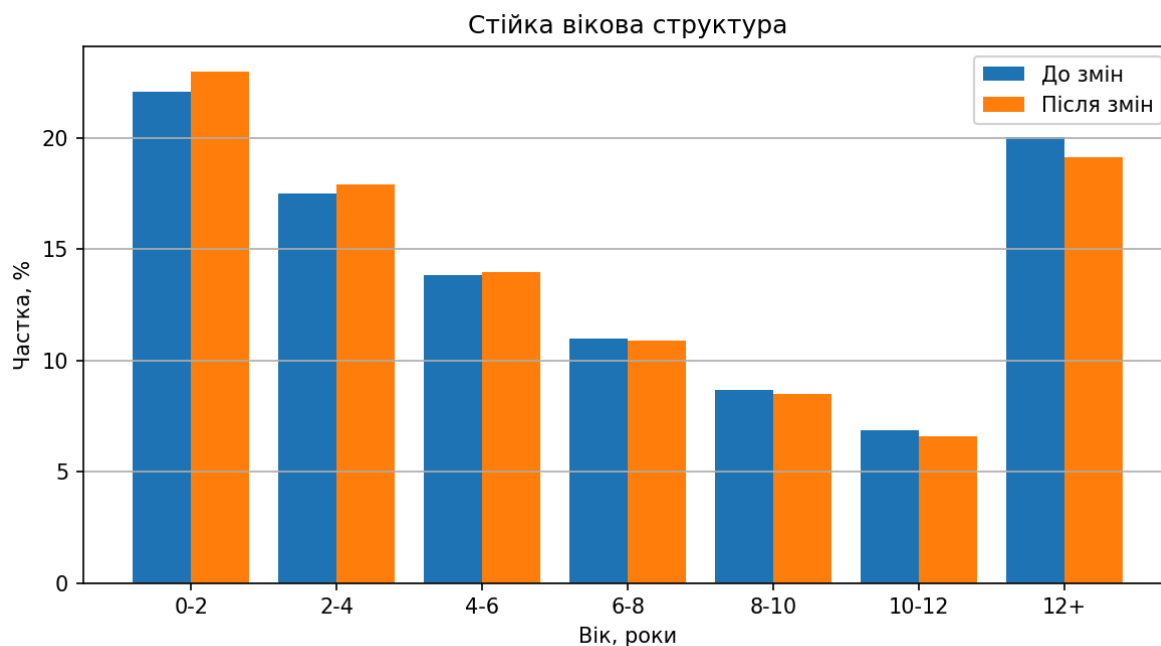
2. Леслі (варіант 2)

7 вікових груп (0–2, ..., 10–12, 12+), крок 2 роки, $b = (0; 0; 0,19; 0,44; 0,5; 0,5; 0,45)$. До змін: переходи 0,87, виживання в старшій групі 0,8. Після змін: переходи 0,8, виживання в старшій групі 0,75. Знайти швидкість росту, стійку вікову структуру, $H = (1 - 1/\lambda_L) \cdot 100 \%$.

Модель: $x(t+1) = L \cdot x(t)$. У першому рядку матриці L – вектор b , на піддіагоналі – коефіцієнти переходу $s_1 \dots s_6$, у правому нижньому куті – s_7 . Швидкість росту – найбільше власне значення λ_L , стійка вікова структура – відповідний власний вектор (у відсотках). Власні значення знайдено в Python (numpy.linalg.eig).

Параметр	До змін	Після змін
λ	1,0986	1,0265
Приріст за 2 роки / за рік	9,86 % / 4,81 %	2,65 % / 1,31 %
H	8,97 %	2,58 %
Структура, % (0–2, 2–4, 4–6, 6–8, 8–10, 10–12, 12+)	22,08; 17,49; 13,85; 10,97; 8,69; 6,88; 20,04	22,99; 17,92; 13,97; 10,89; 8,48; 6,61; 19,13

Отже, в обох випадках $\lambda > 1$ – популяція зростає, але після змін темп росту падає майже в 4 рази, тому допустимий вилов H зменшується з 8,97 % до 2,58 %. Вікова структура змінюється мало: трохи більше наймолодших (22,08 → 22,99 %) і трохи менше старших (20,04 → 19,13 %).



Фрагмент коду і результат його запуску:

```
import numpy as np
import matplotlib.pyplot as plt

# Задача 2. Модель Леслі, варіант 2
# 7 вікових груп (0-2, 2-4, ..., 10-12, 12+), крок 2 роки
b = [0, 0, 0.19, 0.44, 0.5, 0.5, 0.45] # вектор народжуваності

# Матриця Леслі: перший рядок – народжуваність, піддіагональ – коефіцієнти
# переходу в наступну групу, у правому нижньому куті – виживання в старшій групі
def matrix(s, s_last):
    L = np.zeros((7, 7))
    L[0, :] = b
    for i in range(6):
        L[i + 1, i] = s
    L[6, 6] = s_last
    return L

# швидкість росту = найбільше власне значення lambda,
# стійка вікова структура = відповідний власний вектор
def solve(L):
    values, vectors = np.linalg.eig(L)
    k = np.argmax(values.real)
    lam = values[k].real
    x = np.abs(vectors[:, k].real)
    x = x / x.sum() * 100 # переводимо у відсотки
    return lam, x

L1 = matrix(0.87, 0.8) # початкові умови
L2 = matrix(0.8, 0.75) # після змін
```

```
res = []
for name, L in (("До змін", L1), ("Після змін", L2)):
    lam, x = solve(L)
    H = (1 - 1 / lam) * 100 # частка, яку можна виловити
    print(name)
    print(L)
    print("lambda =", round(lam, 5))
    print("приріст за 2 роки:", round((lam - 1) * 100, 2), "%")
    print("приріст за рік:", round((np.sqrt(lam) - 1) * 100, 2), "%")
    print("структура, %:", np.round(x, 2))
    print("перевірка L*x - lam*x:", np.abs(L @ x - lam * x).max().round(10))
    print("H =", round(H, 2), "%")
    print()
    res.append(x)
```

```
dunnaya@MacBook-Air-Dana mod1 % python3 task2.py
До змін
[[0.  0.  0.19 0.44 0.5  0.5  0.45]
 [0.87 0.  0.  0.  0.  0.  0. ]
 [0.  0.87 0.  0.  0.  0.  0. ]
 [0.  0.  0.87 0.  0.  0.  0. ]
 [0.  0.  0.  0.87 0.  0.  0. ]
 [0.  0.  0.  0.  0.87 0.  0. ]
 [0.  0.  0.  0.  0.  0.87 0.8 ]]
lambda = 1.09857
приріст за 2 роки: 9.86 %
приріст за рік: 4.81 %
структура, %: [22.08 17.49 13.85 10.97  8.69  6.88 20.04]
перевірка L*x - lam*x: 0.0
H = 8.97 %
```

```
Після змін
[[0.  0.  0.19 0.44 0.5  0.5  0.45]
 [0.8  0.  0.  0.  0.  0.  0. ]
 [0.  0.8 0.  0.  0.  0.  0. ]
 [0.  0.  0.8 0.  0.  0.  0. ]
 [0.  0.  0.  0.8 0.  0.  0. ]
 [0.  0.  0.  0.  0.8 0.  0. ]
 [0.  0.  0.  0.  0.  0.8 0.75]]
lambda = 1.02646
приріст за 2 роки: 2.65 %
приріст за рік: 1.31 %
структура, %: [22.99 17.92 13.97 10.89  8.48  6.61 19.13]
перевірка L*x - lam*x: 0.0
H = 2.58 %
```

3. Хижак-жертва (варіант 3)

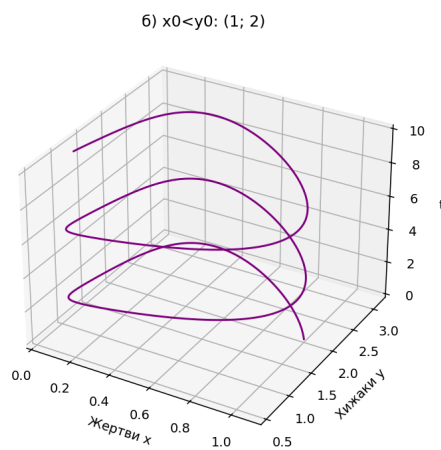
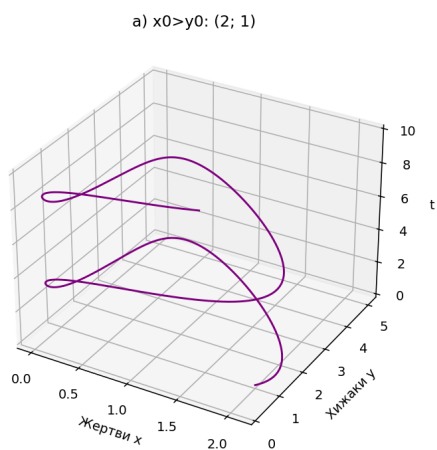
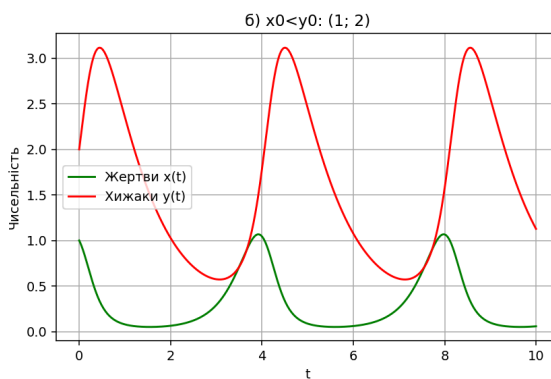
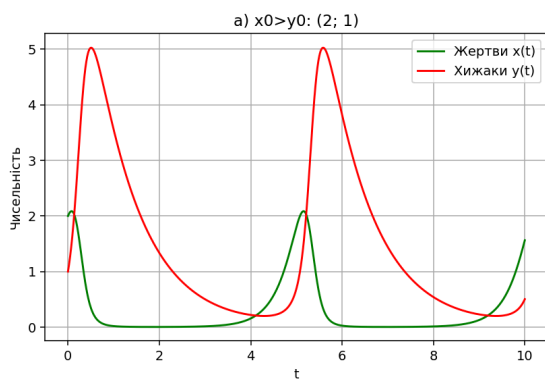
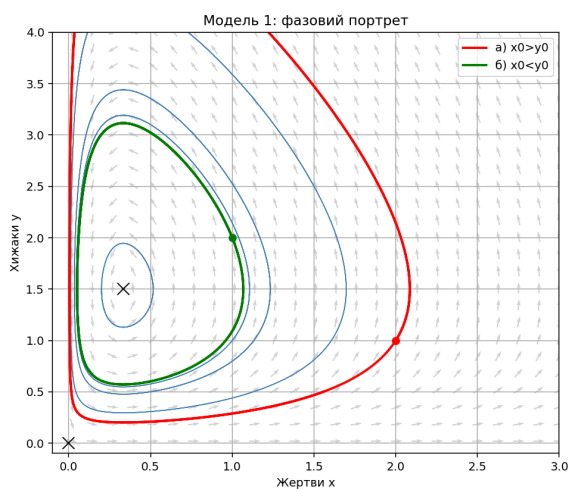
$\varepsilon_1 = 3$ (розмноження жертв), $\varepsilon_2 = 1$ (загибель хижаків), $\gamma_1 = 2$ (жертви зменшуються), $\gamma_2 = 3$ (хижаки зростають), $\beta = 0,5$ (конкуренція жертв). Початкові умови: а) (2; 1), б) (1; 2).

1) Модель Вольтерра

$$x' = x(3 - 2y), \quad y' = y(-1 + 3x).$$

Стационарні точки: $O(0; 0)$ і $P(1/3; 3/2)$. Якобіан $A = [3 - 2y, -2x; 3y, -1 + 3x]$.

- $O(0; 0)$: $\lambda = 3$ та -1 – сідло (нестійка).
- $P(1/3; 3/2)$: $\lambda^2 + 3 = 0$, $\lambda = \pm i\sqrt{3}$ – центр, замкнені траєкторії, незгасаючі коливання.

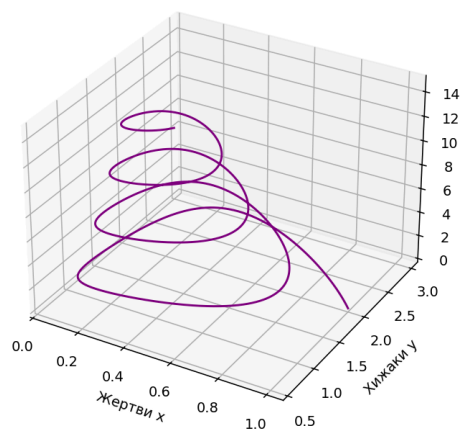
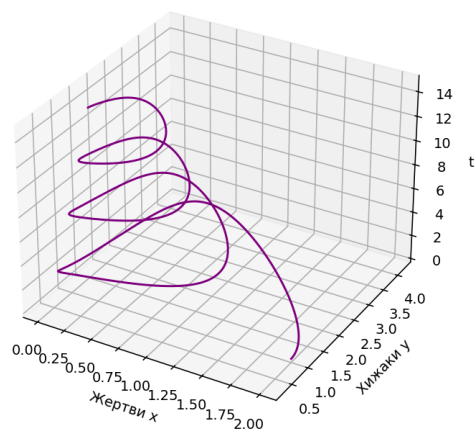
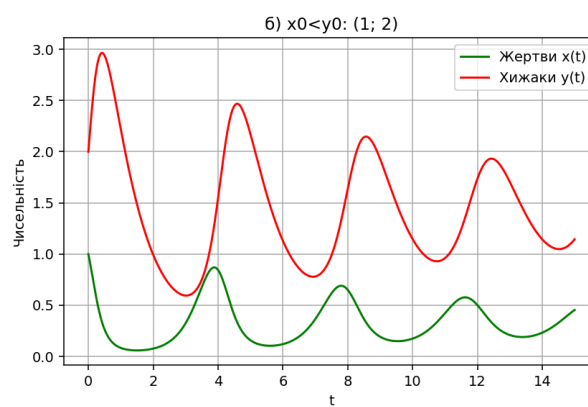
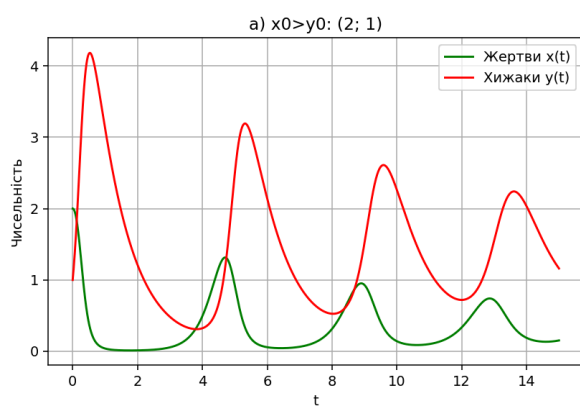
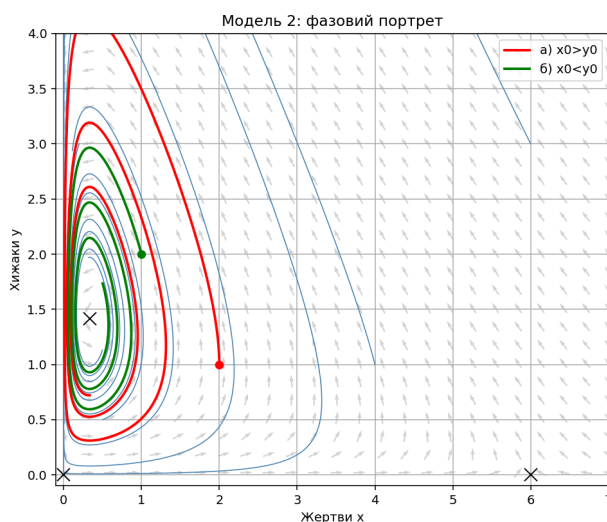


2) Модель Лотки-Вольтерра

$$x' = x(3 - 2y - 0,5x), \quad y' = y(-1 + 3x).$$

Стационарні точки: $O(0; 0)$, $A(6; 0)$, $P(1/3; 17/12)$. Якобіан $A = [3 - 2y - x, -2x; 3y, -1 + 3x]$.

- $O(0; 0)$: $\lambda = 3$ та -1 – сідло.
- $A(6; 0)$: $\lambda = -3$ та 17 – сідло.
- $P(1/3; 17/12)$: $\lambda^2 + \lambda/6 + 17/6 = 0$, $D < 0$, $\lambda \approx -0,083 \pm 1,68i$ – стійкий фокус.



Отже, коливання затухають, обидві траєкторії наближаються до точки (1/3; 17/12). Внутрішньовидова конкуренція перетворює центр на стійкий фокус.

Фрагменти коду і результат їх запуску:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Задача 3. Хижак-жертва, варіант 3
# модель 1:  $x' = x(e_1 - g_1 y)$ ,  $y' = y(-e_2 + g_2 x)$ 
# модель 2:  $x' = x(e_1 - g_1 y - \beta x)$ ,  $y' = y(-e_2 + g_2 x)$ 
e1 = 3      # розмноження жертв
e2 = 1      # загибель хижаків
g1 = 2      # жертви зменшуються
g2 = 3      # хижаки зростають
beta = 0.5  # конкуренція серед жертв

# ----- стаціонарні точки і їх тип -----
# точки знайшли вручну з умови  $x' = 0$ ,  $y' = 0$ :
# (0, 0); (e2/g2, e1/g1) для моделі 1; ще (e1/beta, 0) для моделі 2,
# де в моделі 2  $y = (e_1 g_2 - \beta e_2) / (g_1 g_2)$ 
points1 = [(0, 0), (e2 / g2, e1 / g1)]
points2 = [(0, 0), (e1 / beta, 0), (e2 / g2, (e1 * g2 - beta * e2) / (g1 * g2))]

# матриці Якобі
def jac1(x, y):
    return np.array([[e1 - g1 * y, -g1 * x],
                    [g2 * y, -e2 + g2 * x]])

def jac2(x, y):
    return np.array([[e1 - g1 * y - beta * x, -g1 * x],
                    [g2 * y, -e2 + g2 * x]])
```

```
# тип точки за власними значеннями матриці Якобі
def point_type(ev):
    re = ev.real
    im = ev.imag
    if abs(im).max() < 1e-9:          # власні значення дійсні
        if re.max() > 0 and re.min() < 0:
            return "сідло (нестійка)"
        if re.max() < 0:
            return "стійкий вузол"
        return "нестійкий вузол"
    if abs(re).max() < 1e-9:          # суто уявні
        return "центр"
    if re.max() < 0:
        return "стійкий фокус"
    return "нестійкий фокус"

for name, points, jac in (("Модель 1", points1, jac1), ("Модель 2", points2, jac2)):
    print(name)
    for px, py in points:
        ev = np.linalg.eigvals(jac(px, py))
        print("точка", (round(px, 4), round(py, 4)), " власні значення:", np.round(ev, 4), " -", point_type(ev))
    print()
```

```
dunnaya@MacBook-Air-Dana mod1 % python3 task3.py
Модель 1
точка (0, 0) власні значення: [ 3. -1.] - сідло (нестійка)
точка (0.3333, 1.5) власні значення: [0.+1.7321j 0.-1.7321j] - центр

Модель 2
точка (0, 0) власні значення: [ 3. -1.] - сідло (нестійка)
точка (6.0, 0) власні значення: [-3. 17.] - сідло (нестійка)
точка (0.3333, 1.4167) власні значення: [-0.0833+1.6812j -0.0833-1.6812j] - стійкий фокус

модель 1, старт (2, 1) H max-min = 8.615036684034294e-09
модель 1, старт (1, 2) H max-min = 1.9383890048629837e-09
модель 2, старт (2, 1) кінець: 0.3333 1.4167
модель 2, старт (1, 2) кінець: 0.3333 1.4167
```